

Taller Semana 11

Smart Campus Request Router

Message Routing y Message Transformation con RabbitMQ y Apache Camel

1. Contexto del taller

Una institución educativa está implementando una arquitectura de integración basada en mensajería para procesar solicitudes generadas por diferentes canales digitales.

Actualmente, todas las solicitudes llegan a una cola común llamada:

```
campus.requests.in
```

Sin embargo, no todas las solicitudes deben ser procesadas por el mismo sistema. Dependiendo del tipo de solicitud, el mensaje debe ser enviado a una cola diferente.

Además, los mensajes llegan en un formato externo que no necesariamente coincide con el formato interno que la institución desea utilizar para integrar sus sistemas.

Por esta razón, se requiere construir una solución de integración que sea capaz de:

1. Recibir mensajes desde una cola de entrada.
2. Transformar el mensaje recibido a un formato canónico interno.
3. Enrutar el mensaje transformado hacia la cola correspondiente según su contenido.
4. Enviar mensajes no reconocidos o inválidos a una cola de revisión manual.

Este taller aplica dos patrones fundamentales de integración mediante mensajería:

Patrón	Propósito en el taller
Message Translator	Transformar el mensaje externo en un formato interno común.
Content-Based Router	Enrutar el mensaje según el valor de un campo del contenido.

2. Objetivo del taller

Implementar una ruta de integración con Apache Camel y RabbitMQ que reciba solicitudes estudiantiles, las transforme a un formato canónico y las enrute hacia diferentes colas según el tipo de solicitud.

3. Resultados esperados de aprendizaje

Al finalizar el taller, el estudiante será capaz de:

1. Explicar la diferencia entre enrutar y transformar mensajes.
2. Implementar una ruta de integración usando Apache Camel.
3. Conectar Apache Camel con RabbitMQ.
4. Aplicar el patrón Content-Based Router usando reglas basadas en el contenido del mensaje.
5. Aplicar el patrón Message Translator para convertir un mensaje externo en un mensaje canónico.
6. Evidenciar el funcionamiento de una solución de mensajería mediante pruebas y capturas.
7. Reflexionar sobre el uso de modelos canónicos para reducir el acoplamiento entre sistemas.

4. Modalidad de trabajo

El taller debe realizarse en grupos.

Cada grupo deberá organizarse internamente asignando, como mínimo, los siguientes roles:

Rol	Responsabilidad
Arquitecto de integración	Diseñar el flujo general, identificar patrones y definir las colas.
Desarrollador Camel	Implementar la ruta de integración en Apache Camel.
Responsable RabbitMQ	Configurar RabbitMQ, exchange, colas y pruebas de mensajería.
Responsable de documentación	Elaborar README, evidencias y reflexión técnica.

Un mismo estudiante puede asumir más de un rol si el grupo lo considera necesario.

5. Herramientas requeridas

Para realizar el taller se requiere tener instalado o disponible:

1. Docker.
2. Java JDK 17 o superior.
3. Maven.

4. Un IDE de desarrollo, por ejemplo IntelliJ IDEA, Visual Studio Code o Eclipse.
5. Navegador web.
6. Terminal o consola.
7. Opcional: `jq`, para ejecutar el script automático de publicación de mensajes.

La solución se implementará usando:

- Java.
- Spring Boot.
- Apache Camel.
- RabbitMQ.
- Docker.

6. Caso de negocio

La institución recibe solicitudes de estudiantes desde diferentes canales:

- Formulario web.
- Aplicación móvil.
- Plataforma administrativa.
- Integraciones futuras con sistemas externos.

Todas las solicitudes llegan inicialmente a la cola:

```
campus.requests.in
```

Cada solicitud incluye un campo llamado:

```
request_type
```

Ese campo indica el tipo de solicitud que se debe procesar.

La solución debe enrutar los mensajes de acuerdo con la siguiente regla:

Valor de <code>request_type</code>	Cola destino
ADMISSION	<code>campus.admissions.queue</code>
PAYMENT	<code>campus.payments.queue</code>
SUPPORT	<code>campus.support.queue</code>
ACADEMIC	<code>campus.academic.queue</code>
Otro valor o mensaje inválido	<code>campus.manual-review.queue</code>

7. Mensaje de entrada

El mensaje original que llega a la cola de entrada tendrá el siguiente formato:

```
{
  "request_id": "REQ-1001",
  "student_name": "Ana Pérez",
  "student_document": "1712345678",
  "request_type": "ADMISSION",
  "channel": "web",
  "created_at": "2026-06-10T10:30:00"
}
```

8. Mensaje canónico esperado

Antes de enrutar el mensaje, la solución debe transformarlo al siguiente formato interno:

```
{
  "requestId": "REQ-1001",
  "student": {
    "fullName": "Ana Pérez",
    "document": "1712345678"
  },
  "type": "ADMISSION",
  "sourceChannel": "web",
  "createdAt": "2026-06-10T10:30:00"
}
```

La transformación debe realizar los siguientes cambios:

Campo original	Campo canónico
request_id	requestId
student_name	student.fullName
student_document	student.document
request_type	type
channel	sourceChannel
created_at	createdAt

9. Alcance mínimo esperado

El grupo debe entregar una solución funcional que cumpla con el siguiente alcance mínimo:

1. RabbitMQ ejecutándose correctamente en Docker.
2. Exchange creado.
3. Cola de entrada creada.
4. Cinco colas destino creadas.

5. Ruta Camel que consuma mensajes desde `campus.requests.in`.
6. Transformación del mensaje de entrada al formato canónico.
7. Enrutamiento según el valor del campo `type`.
8. Envío de mensajes no reconocidos a `campus.manual-review.queue`.
9. Envío de mensajes inválidos a `campus.manual-review.queue`.
10. README técnico con explicación clara de la solución.
11. Evidencias de ejecución.

10. Alcance recomendado

Además del alcance mínimo, se recomienda que el grupo implemente:

1. Logs en cada etapa del flujo.
2. Validación básica de campos obligatorios.
3. Separación del código en clases o componentes claros.
4. Al menos un ejemplo de prueba por cada tipo de solicitud.
5. Diagrama del flujo de integración.
6. Explicación explícita de los patrones utilizados.
7. Reflexión sobre cómo extender la solución ante nuevos tipos de solicitud.

11. Actividad previa de investigación

Antes o durante el desarrollo del taller, cada grupo debe investigar brevemente los siguientes temas.

Las respuestas deben incluirse en el README técnico.

11.1 Content-Based Router

Responder:

1. ¿Qué problema resuelve este patrón?
2. ¿Por qué es mejor que el productor no conozca todos los posibles destinos?
3. ¿Qué campo del mensaje se utilizará como criterio de decisión en este taller?

11.2 Message Translator

Responder:

1. ¿Qué problema resuelve este patrón?
2. ¿Por qué dos sistemas pueden necesitar formatos distintos para representar la misma información?
3. ¿Qué transformación concreta se realizará en este taller?

11.3 Canonical Data Model

Responder:

1. ¿Qué es un modelo canónico?
2. ¿Por qué puede reducir el acoplamiento entre sistemas?
3. ¿Cuál es el modelo canónico definido para este taller?

11.4 RabbitMQ

Responder:

1. ¿Qué es una cola en RabbitMQ?
2. ¿Qué diferencia existe entre exchange, queue y routing key?
3. ¿Cómo se puede verificar que un mensaje llegó correctamente a una cola?

12. Desarrollo paso a paso tipo tutorial

En esta sección se indican los comandos, archivos y fragmentos de código necesarios para construir el taller.

El objetivo es que el grupo pueda seguir una guía práctica, ejecutar la solución y comprender qué parte del código implementa cada patrón de integración.

Paso 1. Crear la carpeta del proyecto

Crear una carpeta para el taller:

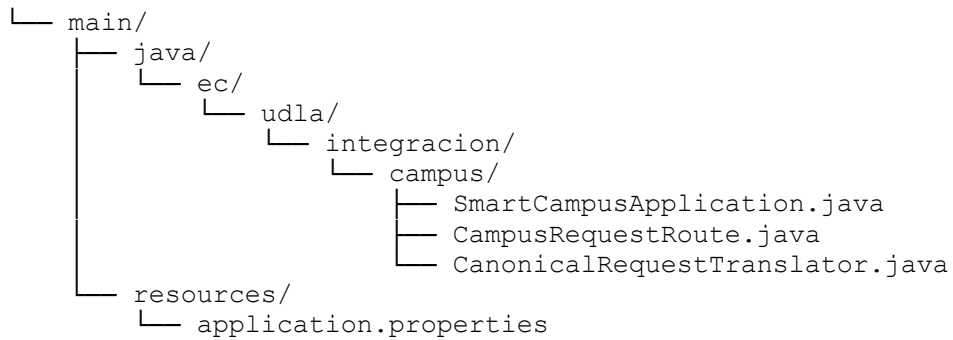
```
mkdir smart-campus-request-router
cd smart-campus-request-router
```

Crear la estructura base del proyecto:

```
mkdir -p src/main/java/ec/udla/integracion/campus
mkdir -p src/main/resources
mkdir -p scripts
```

La estructura esperada será:

```
smart-campus-request-router/
├── pom.xml
├── docker-compose.yml
├── scripts/
│   ├── setup-rabbitmq.sh
│   └── publish-messages.sh
└── src/
```



Paso 2. Crear el archivo `docker-compose.yml`

En la raíz del proyecto, crear el archivo:

```
touch docker-compose.yml
```

Agregar el siguiente contenido:

```
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq-campus
    hostname: rabbitmq-campus
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
```

Levantar RabbitMQ:

```
docker compose up -d
```

Verificar que el contenedor esté corriendo:

```
docker ps
```

Ingresar a la consola de administración de RabbitMQ desde el navegador:

```
http://localhost:15672
```

Credenciales:

Usuario: guest
Contraseña: guest

Paso 3. Crear exchange, colas y bindings en RabbitMQ

Crear el archivo:

```
touch scripts/setup-rabbitmq.sh
```

Agregar el siguiente contenido:

```
#!/bin/bash

RABBITMQ_API="http://localhost:15672/api"
USER="guest"
PASS="guest"
VHOST="%2F"
EXCHANGE="campus.exchange"

echo "Creando exchange..."

curl -u $USER:$PASS -H "content-type:application/json" \
  -X PUT $RABBITMQ_API/exchanges/$VHOST/$EXCHANGE \
  -d '{"type":"direct","durable":true}'

echo ""
echo "Creando colas..."

QUEUES=(
  "campus.requests.in"
  "campus.admissions.queue"
  "campus.payments.queue"
  "campus.support.queue"
  "campus.academic.queue"
  "campus.manual-review.queue"
)

for QUEUE in "${QUEUES[@]}"
do
  curl -u $USER:$PASS -H "content-type:application/json" \
    -X PUT $RABBITMQ_API/queues/$VHOST/$QUEUE \
    -d '{"durable":true}'

  echo ""
  echo "Cola creada: $QUEUE"
done

echo ""
echo "Creando bindings..."

for QUEUE in "${QUEUES[@]}"
do
  curl -u $USER:$PASS -H "content-type:application/json" \
    -X POST $RABBITMQ_API/bindings/$VHOST/e/$EXCHANGE/q/$QUEUE \
    -d '{"routing_key":"$QUEUE"}'

  echo ""
  echo "Binding creado para routing key: $QUEUE"
done

echo ""
echo "Configuración de RabbitMQ completada."
```


Dar permisos de ejecución:

```
chmod +x scripts/setup-rabbitmq.sh
```

Ejecutar el script:

```
./scripts/setup-rabbitmq.sh
```

Luego ingresar a RabbitMQ Management UI y verificar que existan estas colas:

```
campus.requests.in  
campus.admissions.queue  
campus.payments.queue  
campus.support.queue  
campus.academic.queue  
campus.manual-review.queue
```

Paso 4. Crear el archivo `pom.xml`

En la raíz del proyecto, crear el archivo:

```
touch pom.xml
```

Agregar el siguiente contenido:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
                                https://maven.apache.org/xsd/maven-  
4.0.0.xsd">  
  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>ec.udla.integracion</groupId>  
    <artifactId>smart-campus-request-router</artifactId>  
    <version>1.0.0</version>  
  
    <properties>  
        <java.version>17</java.version>  
        <spring-boot.version>3.3.5</spring-boot.version>  
        <camel.version>4.8.1</camel.version>  
    </properties>  
  
    <dependencyManagement>  
        <dependencies>  
            <dependency>  
                <groupId>org.springframework.boot</groupId>  
                <artifactId>spring-boot-dependencies</artifactId>  
                <version>${spring-boot.version}</version>  
                <type>pom</type>  
                <scope>import</scope>  
            </dependency>  
  
            <dependency>  
                <groupId>org.apache.camel.springboot</groupId>  
                <artifactId>camel-spring-boot-bom</artifactId>
```

```

        <version>${camel.version}</version>
        <type>pom</type>
        <scope>import</scope>
    </dependency>
</dependencies>
</dependencyManagement>

<dependencies>
    <!-- Spring Boot base -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
    </dependency>

    <!-- Apache Camel con Spring Boot -->
    <dependency>
        <groupId>org.apache.camel.springboot</groupId>
        <artifactId>camel-spring-boot-starter</artifactId>
    </dependency>

    <!-- Integración Camel con RabbitMQ usando Spring RabbitMQ -->
    <dependency>
        <groupId>org.apache.camel.springboot</groupId>
        <artifactId>camel-spring-rabbitmq-starter</artifactId>
    </dependency>

    <!-- Soporte JSON con Jackson -->
    <dependency>
        <groupId>org.apache.camel.springboot</groupId>
        <artifactId>camel-jackson-starter</artifactId>
    </dependency>

    <!-- Soporte para JSONPath en Camel -->
    <dependency>
        <groupId>org.apache.camel.springboot</groupId>
        <artifactId>camel-jsonpath-starter</artifactId>
    </dependency>

    <!-- Utilidad para manejo de JSON -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
    </dependency>
</dependencies>

<build>
    <plugins>
        <!-- Plugin para ejecutar Spring Boot -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <version>${spring-boot.version}</version>
        </plugin>
    </plugins>
</build>
</project>

```

Paso 5. Crear el archivo `application.properties`

Crear el archivo:

```
touch src/main/resources/application.properties
```

Agregar el siguiente contenido:

```
spring.application.name=smart-campus-request-router

# Configuración de RabbitMQ
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

# Mantener la aplicación Camel ejecutándose
camel.springboot.main-run-controller=true

# Logs
logging.level.org.apache.camel=INFO
logging.level.ec.udla.integracion.campus=INFO
```

Paso 6. Crear la clase principal de Spring Boot

Crear el archivo:

```
touch
src/main/java/ec/udla/integracion/campus/SmartCampusApplication.java
```

Agregar el siguiente contenido:

```
package ec.udla.integracion.campus;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SmartCampusApplication {

    public static void main(String[] args) {
        SpringApplication.run(SmartCampusApplication.class, args);
    }
}
```

Esta clase permite iniciar la aplicación Spring Boot que ejecutará las rutas de Apache Camel.

Paso 7. Crear el Message Translator

Crear el archivo:

```
touch
src/main/java/ec/udla/integracion/campus/CanonicalRequestTranslator.java
```

Agregar el siguiente contenido:

```
package ec.udla.integracion.campus;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;
import org.apache.camel.Exchange;
import org.apache.camel.Processor;
import org.springframework.stereotype.Component;

@Component
public class CanonicalRequestTranslator implements Processor {

    private final ObjectMapper objectMapper = new ObjectMapper();

    @Override
    public void process(Exchange exchange) throws Exception {

        String originalMessage =
exchange.getIn().getBody(String.class);
        JsonNode originalJson =
objectMapper.readTree(originalMessage);

        boolean isValid =
            hasText(originalJson, "request_id") &&
            hasText(originalJson, "student_name") &&
            hasText(originalJson, "student_document") &&
            hasText(originalJson, "request_type") &&
            hasText(originalJson, "channel") &&
            hasText(originalJson, "created_at");

        ObjectNode canonicalJson = objectMapper.createObjectNode();

        if (!isValid) {
            canonicalJson.put("status", "INVALID");
            canonicalJson.put("reason", "Mensaje incompleto o con
campos obligatorios ausentes");
            canonicalJson.set("originalMessage", originalJson);

            exchange.setProperty("requestType", "INVALID");

exchange.getIn().setBody(objectMapper.writeValueAsString(canonicalJson
));
            return;
        }

        String requestType =
originalJson.get("request_type").asText();

        ObjectNode student = objectMapper.createObjectNode();
```

```

        student.put("fullName",
originalJson.get("student_name").asText());
        student.put("document",
originalJson.get("student_document").asText());

        canonicalJson.put("requestId",
originalJson.get("request_id").asText());
        canonicalJson.set("student", student);
        canonicalJson.put("type", requestType);
        canonicalJson.put("sourceChannel",
originalJson.get("channel").asText());
        canonicalJson.put("createdAt",
originalJson.get("created_at").asText());

        exchange.setProperty("requestType", requestType);

exchange.getIn().setBody(objectMapper.writeValueAsString(canonicalJson
));
    }

    private boolean hasText(JsonNode jsonNode, String fieldName) {
        return jsonNode.has(fieldName)
            && !jsonNode.get(fieldName).isNull()
            && !jsonNode.get(fieldName).asText().isBlank();
    }
}

```

Este componente implementa el patrón **Message Translator**, porque convierte el mensaje externo recibido por la cola de entrada en un formato canónico interno.

Ejemplo de entrada:

```

{
  "request_id": "REQ-1001",
  "student_name": "Ana Pérez",
  "student_document": "1712345678",
  "request_type": "ADMISSION",
  "channel": "web",
  "created_at": "2026-06-10T10:30:00"
}

```

Ejemplo de salida canónica:

```

{
  "requestId": "REQ-1001",
  "student": {
    "fullName": "Ana Pérez",
    "document": "1712345678"
  },
  "type": "ADMISSION",
  "sourceChannel": "web",
  "createdAt": "2026-06-10T10:30:00"
}

```

Paso 8. Crear la ruta Camel con Content-Based Router

Crear el archivo:

```
touch src/main/java/ec/udla/integracion/campus/CampusRequestRoute.java
```

Agregar el siguiente contenido:

```
package ec.udla.integracion.campus;

import org.apache.camel.builder.RouteBuilder;
import org.springframework.stereotype.Component;

@Component
public class CampusRequestRoute extends RouteBuilder {

    private final CanonicalRequestTranslator
canonicalRequestTranslator;

    public CampusRequestRoute(CanonicalRequestTranslator
canonicalRequestTranslator) {
        this.canonicalRequestTranslator = canonicalRequestTranslator;
    }

    @Override
    public void configure() {

        from("spring-rabbitmq:campus.exchange"
            + "?queues=campus.requests.in"
            + "&routingKey=campus.requests.in"
            + "&autoDeclare=false")
            .routeId("smart-campus-request-router")

            .log("Mensaje recibido desde campus.requests.in: ${body}")

            .process(canonicalRequestTranslator)

            .log("Mensaje transformado a formato canónico: ${body}")
            .log("Tipo de solicitud detectado:
${exchangeProperty.requestType}")

            .choice()

            .when(exchangeProperty("requestType").isEqualTo("ADMISSION"))
                .log("Enrutando solicitud de admisión")
                .to("spring-
rabbitmq:campus.exchange?routingKey=campus.admissions.queue")

            .when(exchangeProperty("requestType").isEqualTo("PAYMENT"))
                .log("Enrutando solicitud de pago")
                .to("spring-
rabbitmq:campus.exchange?routingKey=campus.payments.queue")

            .when(exchangeProperty("requestType").isEqualTo("SUPPORT"))
                .log("Enrutando solicitud de soporte")
                .to("spring-
rabbitmq:campus.exchange?routingKey=campus.support.queue")
    }
}
```

```

.when(exchangeProperty("requestType").isEqualTo("ACADEMIC"))
    .log("Enrutando solicitud académica")
    .to("spring-
rabbitmq:campus.exchange?routingKey=campus.academic.queue")

    .otherwise()
    .log("Solicitud no reconocida o inválida. Enviando
a revisión manual")
    .to("spring-
rabbitmq:campus.exchange?routingKey=campus.manual-review.queue")
    .end();
}
}

```

Esta ruta implementa dos patrones:

Message Translator

Se aplica en esta línea:

```
.process(canonicalRequestTranslator)
```

Content-Based Router

Se aplica en este bloque:

```

.choice()
    .when(exchangeProperty("requestType").isEqualTo("ADMISSION"))
    .when(exchangeProperty("requestType").isEqualTo("PAYMENT"))
    .when(exchangeProperty("requestType").isEqualTo("SUPPORT"))
    .when(exchangeProperty("requestType").isEqualTo("ACADEMIC"))
    .otherwise()

```

Paso 9. Compilar el proyecto

Ejecutar:

```
mvn clean package
```

Si la compilación es correcta, se debe observar un mensaje similar a:

```
BUILD SUCCESS
```

Si aparece un error, revisar:

1. Que Java 17 o superior esté instalado.
2. Que Maven esté instalado.
3. Que el archivo `pom.xml` esté en la raíz del proyecto.
4. Que las clases Java estén en el paquete correcto.
5. Que no existan errores de escritura en los nombres de archivos o clases.

Paso 10. Ejecutar la aplicación

Ejecutar:

```
mvn spring-boot:run
```

La aplicación debe quedar en ejecución.

En la consola deberían aparecer logs de Apache Camel indicando que la ruta fue iniciada.

No cerrar esta terminal mientras se realizan las pruebas.

Paso 11. Crear script para publicar mensajes de prueba

Abrir una segunda terminal en la raíz del proyecto.

Crear el archivo:

```
touch scripts/publish-messages.sh
```

Agregar el siguiente contenido:

```
#!/bin/bash

RABBITMQ_API="http://localhost:15672/api"
USER="guest"
PASS="guest"
VHOST="%2F"
EXCHANGE="campus.exchange"
ROUTING_KEY="campus.requests.in"

publish_message() {
    local message="$1"

    curl -u $USER:$PASS -H "content-type:application/json" \
        -X POST $RABBITMQ_API/exchanges/$VHOST/$EXCHANGE/publish \
        -d "{
            \"properties\": {
                \"content_type\": \"application/json\"
            },
            \"routing_key\": \"$ROUTING_KEY\",
            \"payload\": $(echo \"$message\" | jq -Rs .),
            \"payload_encoding\": \"string\"
        }"

    echo ""
    echo "Mensaje publicado:"
    echo "$message"
}
```



```

    echo ""
}

echo "Publicando mensaje ADMISSION..."

publish_message '{
  "request_id": "REQ-1001",
  "student_name": "Ana Pérez",
  "student_document": "1712345678",
  "request_type": "ADMISSION",
  "channel": "web",
  "created_at": "2026-06-10T10:30:00"
}'

echo "Publicando mensaje PAYMENT..."

publish_message '{
  "request_id": "REQ-1002",
  "student_name": "Luis Gómez",
  "student_document": "1722222222",
  "request_type": "PAYMENT",
  "channel": "mobile",
  "created_at": "2026-06-10T11:00:00"
}'

echo "Publicando mensaje SUPPORT..."

publish_message '{
  "request_id": "REQ-1003",
  "student_name": "Carla Torres",
  "student_document": "1733333333",
  "request_type": "SUPPORT",
  "channel": "admin-platform",
  "created_at": "2026-06-10T11:30:00"
}'

echo "Publicando mensaje ACADEMIC..."

publish_message '{
  "request_id": "REQ-1004",
  "student_name": "Pedro Morales",
  "student_document": "1744444444",
  "request_type": "ACADEMIC",
  "channel": "web",
  "created_at": "2026-06-10T12:00:00"
}'

echo "Publicando mensaje con tipo no reconocido..."

publish_message '{
  "request_id": "REQ-1005",
  "student_name": "María Sánchez",
  "student_document": "1755555555",
  "request_type": "LIBRARY",
  "channel": "web",
  "created_at": "2026-06-10T12:30:00"
}'

echo "Publicando mensaje inválido..."

publish_message '{

```

```
"request_id": "REQ-1006",  
"student_name": "Diego Ruiz",  
"channel": "web"  
}'
```

Este script requiere tener instalado `jq`.

En Mac se puede instalar con:

```
brew install jq
```

En Linux basado en Ubuntu se puede instalar con:

```
sudo apt install jq
```

Dar permisos de ejecución:

```
chmod +x scripts/publish-messages.sh
```

Ejecutar el script:

```
./scripts/publish-messages.sh
```

Paso 12. Alternativa si no se tiene `jq`

Si el grupo no tiene `jq`, puede publicar los mensajes manualmente desde RabbitMQ Management UI.

Ingresa a:

```
http://localhost:15672
```

Luego seguir estos pasos:

1. Ir a la pestaña **Exchanges**.
2. Seleccionar `campus.exchange`.
3. Ir a la sección **Publish message**.
4. En `Routing key`, escribir:

```
campus.requests.in
```

5. En `Payload`, pegar uno de los mensajes JSON.
6. Presionar **Publish message**.

Paso 13. Verificar que los mensajes llegaron a la cola correcta

En RabbitMQ Management UI, ingresar a la pestaña **Queues and Streams**.

Verificar lo siguiente:

Mensaje publicado	Cola esperada
ADMISSION	campus.admissions.queue
PAYMENT	campus.payments.queue
SUPPORT	campus.support.queue
ACADEMIC	campus.academic.queue
LIBRARY	campus.manual-review.queue
Mensaje inválido	campus.manual-review.queue

Para revisar el contenido de una cola:

1. Abrir la cola correspondiente.
2. Ir a la sección **Get messages**.
3. En Messages, colocar 1.
4. Presionar **Get Message(s)**.
5. Verificar que el mensaje esté en formato canónico.

Ejemplo de mensaje esperado en `campus.admissions.queue`:

```
{
  "requestId": "REQ-1001",
  "student": {
    "fullName": "Ana Pérez",
    "document": "1712345678"
  },
  "type": "ADMISSION",
  "sourceChannel": "web",
  "createdAt": "2026-06-10T10:30:00"
}
```

Ejemplo de mensaje esperado en `campus.manual-review.queue` para un mensaje inválido:

```
{
  "status": "INVALID",
  "reason": "Mensaje incompleto o con campos obligatorios ausentes",
  "originalMessage": {
    "request_id": "REQ-1006",
    "student_name": "Diego Ruiz",
    "channel": "web"
  }
}
```

Paso 14. Evidenciar logs de Apache Camel

En la terminal donde se está ejecutando la aplicación, verificar que aparezcan logs similares a:

```
Mensaje recibido desde campus.requests.in: {...}  
Mensaje transformado a formato canónico: {...}  
Tipo de solicitud detectado: ADMISSION  
Enrutando solicitud de admisión
```

Tomar captura de pantalla de los logs.

Paso 15. Probar un nuevo tipo de solicitud

Agregar manualmente un nuevo mensaje con tipo SCHOLARSHIP:

```
{  
  "request_id": "REQ-1007",  
  "student_name": "Sofia Andrade",  
  "student_document": "17666666666",  
  "request_type": "SCHOLARSHIP",  
  "channel": "web",  
  "created_at": "2026-06-10T13:00:00"  
}
```

Como SCHOLARSHIP no está contemplado en las reglas actuales, debe ser enviado a:

```
campus.manual-review.queue
```

Luego responder en el README:

```
¿Qué cambios serían necesarios para soportar formalmente el nuevo tipo  
de solicitud SCHOLARSHIP?
```

La respuesta esperada debe mencionar, como mínimo:

1. Crear una nueva cola, por ejemplo `campus.scholarship.queue`.
2. Crear un binding en RabbitMQ.
3. Agregar una nueva condición en el Content-Based Router.
4. Agregar pruebas para el nuevo tipo de mensaje.

Paso 16. Apagar la solución

Cuando finalicen las pruebas, detener la aplicación con:

```
CTRL + C
```

Apagar RabbitMQ:

```
docker compose down
```

Si se desea eliminar también los datos persistidos de RabbitMQ, ejecutar:

```
docker compose down -v
```

13. Entregables

Cada grupo debe entregar los siguientes elementos.

13.1 Código fuente

Debe incluir:

```
pom.xml
docker-compose.yml
scripts/setup-rabbitmq.sh
scripts/publish-messages.sh
src/main/java/ec/udla/integracion/campus/SmartCampusApplication.java
src/main/java/ec/udla/integracion/campus/CampusRequestRoute.java
src/main/java/ec/udla/integracion/campus/CanonicalRequestTranslator.java
src/main/resources/application.properties
README.md
```

No se debe entregar únicamente capturas de pantalla. El código es obligatorio.

13.2 README técnico

El README debe incluir:

1. Nombre del taller.
2. Integrantes del grupo.
3. Descripción del problema de integración.
4. Diagrama del flujo.
5. Tecnologías utilizadas.
6. Instrucciones para ejecutar RabbitMQ.
7. Instrucciones para configurar exchange, colas y bindings.
8. Instrucciones para ejecutar la aplicación.
9. Instrucciones para publicar mensajes de prueba.
10. Tabla con reglas de enrutamiento.
11. Explicación del Message Translator.
12. Explicación del Content-Based Router.

13. Explicación del modelo canónico.
14. Evidencias de ejecución.
15. Problemas encontrados y cómo los resolvieron.
16. Reflexión técnica final.

13.3 Evidencias obligatorias

Incluir capturas de pantalla de:

1. Contenedor RabbitMQ ejecutándose.
2. RabbitMQ Management UI.
3. Exchange `campus.exchange`.
4. Colas creadas.
5. Mensaje publicado en `campus.requests.in`.
6. Mensaje transformado en `campus.admissions.queue`.
7. Mensaje transformado en `campus.payments.queue`.
8. Mensaje transformado en `campus.support.queue`.
9. Mensaje transformado en `campus.academic.queue`.
10. Mensaje no reconocido en `campus.manual-review.queue`.
11. Mensaje inválido en `campus.manual-review.queue`.
12. Logs de Apache Camel.

14. Preguntas de reflexión obligatorias

Responder en el README:

1. ¿Qué problema resuelve el patrón Message Translator en este taller?
2. ¿Qué problema resuelve el patrón Content-Based Router en este taller?
3. ¿Por qué primero se transforma el mensaje y luego se enruta?
4. ¿Qué pasaría si cada productor tuviera que conocer todas las colas destino?
5. ¿Qué ventaja tiene usar un modelo canónico interno?
6. ¿Qué limitaciones tiene esta solución?
7. ¿Cómo se podría mejorar el manejo de errores?
8. ¿Qué cambios serían necesarios para soportar el nuevo tipo de solicitud SCHOLARSHIP?
9. ¿Qué riesgos tendría colocar toda la lógica de decisión en el productor del mensaje?
10. ¿Cómo se relaciona este taller con una arquitectura orientada a eventos?

15. Criterios de evaluación

Criterio	Peso
Configuración correcta de RabbitMQ, exchange, colas y bindings	15%
Proyecto Java/Spring Boot/Camel correctamente estructurado	10%
Implementación correcta del Message Translator	20%
Implementación correcta del Content-Based Router	25%
Pruebas funcionales con mensajes válidos, no reconocidos e inválidos	15%
README, evidencias y reflexión técnica	15%
Total	100%

16. Diagrama de referencia

Antes de programar, el grupo debe dibujar el flujo de integración en una hoja, pizarra o herramienta digital.

El diseño mínimo debería verse así:

```
campus.requests.in
    |
    v
[ Message Translator ]
    |
    v
[ Content-Based Router ]
    |
    |--> campus.admissions.queue
    |--> campus.payments.queue
    |--> campus.support.queue
    |--> campus.academic.queue
    |--> campus.manual-review.queue
```

Primero diseñen el flujo. Luego implementen. Finalmente prueben y documenten.

El objetivo del taller no es solo que el mensaje llegue a una cola, sino que ustedes comprendan cómo una solución de integración puede tomar decisiones y adaptar mensajes sin acoplar directamente a todos los sistemas participantes.